INTERNSHIP PROJECT REPORT

Real-Time Chat Platform

One-to-One Real-Time Messaging Application

Prepared by: Team B

Tech Stack: Django · Django REST Framework · Django Channels · Redis · React · Tailwind CSS · PostgreSQL

Document type: Project Architecture & Planning Report

1. Introduction

1.1 Problem Statement

Many messaging applications are either overloaded with unnecessary features or difficult to customize for educational or organizational use. This project aims to build a scalable, secure, and real-time chat platform that demonstrates modern web application architecture using Django, REST APIs, WebSockets, Redis, and a responsive frontend.

1.2 Objective

To develop a production-style chat application that enables users to communicate instantly while following clean architecture, scalable backend practices, and industry-standard development workflows.

1.3 Target Users

- Students
- Organizations
- Small Teams
- Developers
- Educational Institutions

2. Technology Stack

Layer	Technologies
Backend	Python, Django, Django REST Framework, Django Channels, Redis
Frontend	React, Tailwind CSS, Axios, React Router
Database	PostgreSQL
Deployment	Docker, Nginx, Gunicorn, GitHub Actions (optional)

3. Team Structure & Development Workflow

The project follows a hub-and-spoke workflow led by the Team Lead, with parallel workstreams for Backend, Frontend, Database, and Authentication feeding into a WebSocket integration stage, followed by testing, DevOps, and documentation before final submission.

- Team Lead oversees Backend, Frontend, Database, and Authentication teams
- These four teams converge with the WebSocket Team
- Team Lead performs final integration of all modules
- Testing, DevOps, and Documentation are handled as a combined final stage
- Project concludes with Final Submission

3.1 GitHub Branch Structure

- main — production-ready code

- develop — integration branch
- backend/*, frontend/*, database/*, auth/*, websocket/*, testing/*, docs/*, deployment/* — module branches
- Feature branches: feature/login, feature/signup, feature/chat-ui, feature/chat-api, feature/message-model, feature/typing-status, feature/profile, feature/search

3.2 Repository Folder Structure

Folder	Purpose
backend/apps/, config/, requirements.txt, manage.py	Django backend application and configuration
frontend/src/, public/, package.json	React frontend application
docs/	Project documentation (README, API docs)
assets/	Shared static assets
docker/, docker-compose.yml	Containerization and deployment configuration
.github/	CI/CD workflow configuration

3.3 Team Responsibilities & Deliverables

Team	Prep Work	Deliverables
Backend	API documentation, business rules, folder structure, coding standards	Models, Views, Serializers, URLs, Services, Tests
Frontend	Figma, API endpoints, color palette, components list	Pages, Components, Layouts, State Management, API Integration
Database	ER diagram, relationships, constraints	Models, Migrations, Indexes, Optimization
Authentication	Auth flow, user roles, permission matrix	Login, Signup, JWT/Session, Permissions
WebSocket	Real-time event flow, channel architecture, Redis setup	Consumers, Routing, Notifications, Presence, Typing, Read Receipts
Testing / DevOps / Docs	Test plans, Docker, environment config	Unit & integration tests, bug reports, CI/CD, README, API docs, architecture diagrams, user & installation guides, final report

4. API Documentation

The following endpoints form the initial contract shared between backend and frontend, allowing both teams to work in parallel:

Method	Endpoint	Purpose
POST	/login	Authenticate a user
POST	/signup	Register a new user
GET	/users	List users
GET	/profile	Get current user profile
GET	/conversations	List conversations
GET	/messages	Retrieve message history
POST	/messages	Send a new message
PUT	/messages/{id}	Edit a message
DELETE	/messages/{id}	Delete a message

5. UI Design

Frontend development is guided by a shared design system so all developers build consistent screens rather than inventing their own designs:

- Figma design or wireframes
- Color palette
- Typography
- Component library
- Icons
- Responsive layouts

6. User Roles

The system keeps roles simple in its initial version.

6.1 User

- Register
- Login
- Chat
- Upload profile
- Search users

6.2 Admin

- Manage users
- Suspend accounts
- View reports

- Monitor activity

6.3 User Flow

User Opens Website → Login / Register → Dashboard → Search Users → Open Conversation → Load Previous Messages → Connect WebSocket → Send Message (stored in Database) → Notify Receiver → Receiver Loads Message.

7. MVP Features

These are the core features all teams agree to build first.

Module	Features
Authentication	Registration, Login, Logout, Password Reset, JWT/Session Auth, Protected Routes
User Management	Profile, Edit Profile, Change Password, Avatar Upload
Chat	One-to-One Chat, Real-Time Messaging, Conversation List, Message History, Timestamps
Real-Time Features	WebSocket Connection, Online Status, Typing Indicator, Read Receipts, Delivery Status
Search	Search Users, Search Conversations
Notifications	New Message Notification, Browser Notification (optional)
Media	Image Sharing, File Sharing
Settings	Dark Mode, Notification Preferences

7.1 Feature Breakdown by Module

Module	Features
Authentication	Login, Register, Logout
Profile	Avatar, Bio, Status
Chat	One-to-One Chat
Messages	Send, Receive, History
Search	User Search
WebSocket	Live Updates
Notifications	Message Alerts
Settings	Theme, Preferences

Module	Features
Admin	User Management

7.2 Future Features (Not in MVP)

These ideas are documented for future scope but are not assigned in the initial build:

- Group Chat
- Voice Messages
- Video Calls
- Message Reactions
- Pinned Chats
- Scheduled Messages
- Message Editing / Deletion
- AI Chat Assistant
- End-to-End Encryption
- Push Notifications
- Message Search by Content

8. Requirements

8.1 Functional Requirements

- **Authentication:** the system shall allow registration, secure login, logout, and protect authenticated routes.
- **Messaging:** the system shall send messages in real time, store every message, load chat history, display timestamps, and support media attachments.
- **User Management:** the system shall allow profile updates, display online status, and show profile pictures.
- **Notifications:** the system shall notify users of new messages, update chat previews, and display unread message counts.

8.2 Non-Functional Requirements

Category	Requirements
Performance	Low latency for messaging, fast API responses, efficient database queries
Security	Password hashing, secure authentication, input validation, CSRF/XSS protection, permission checks
Scalability	Modular architecture, Redis-backed WebSockets, API versioning support, clean service separation
Maintainability	Consistent coding standards, modular apps, reusable components, clear documentation

9. Project Milestones

Milestone	Goal
Milestone 1	Project setup, GitHub, architecture
Milestone 2	Authentication complete
Milestone 3	Database and models complete
Milestone 4	REST APIs complete
Milestone 5	Frontend integration
Milestone 6	WebSocket integration
Milestone 7	Testing and bug fixing
Milestone 8	Documentation and deployment

10. Testing & QA Findings

During manual testing of the chat platform's current build, the following issues were identified. These findings should be logged as bug tickets and triaged before the next milestone.

10.1 Message Display & Ordering

- **Issue 1 — Message order reversed:** the latest message appears at the top of the chat screen. As in WhatsApp, the most recent message from another user should appear at the bottom of the conversation.

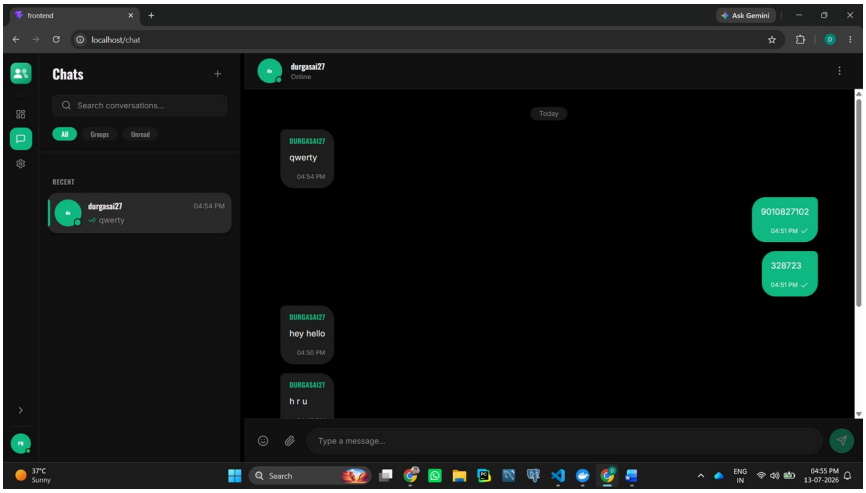

Figure 1: Chat screen showing latest message positioned at the top.

- **Issue 2 — Intermittent display failure:** the latest message sometimes does not display correctly in the conversation view.
- **Issue 3 — Dashboard preview not updating:** the recent chats list on the dashboard page does not show the latest message for a conversation.

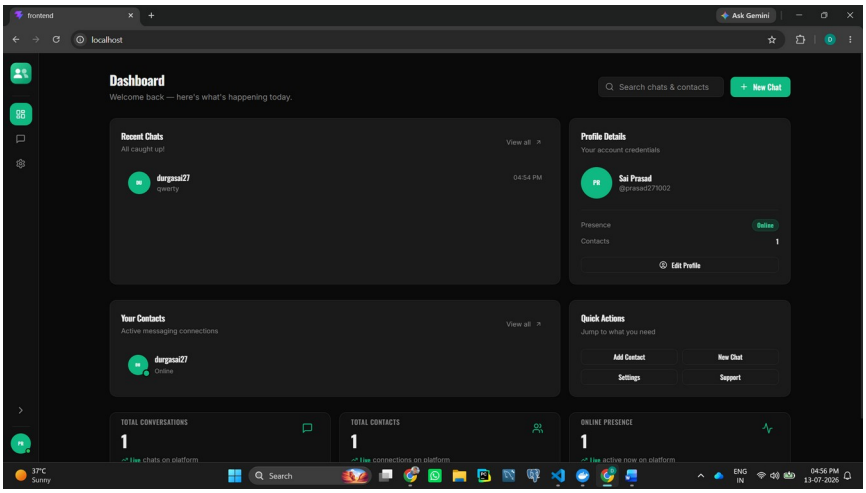

Figure 2: Dashboard recent chats list not reflecting the latest message.

10.2 Navigation & Chat Loading

- **Issue 4 — Chat clears on user selection:** clicking a user's name in the left-hand chat list (http://localhost/chat) erases the chat view and shows it as empty. Refreshing the page restores the existing chat history.

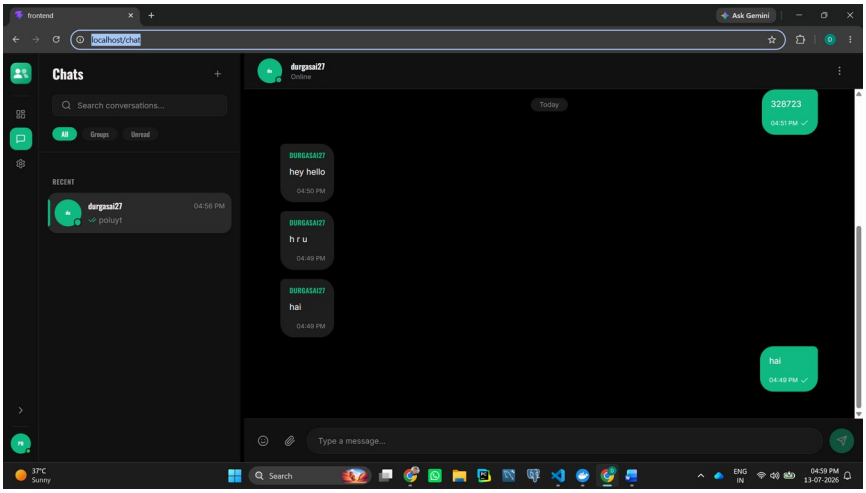

Figure 3: Chat view showing as empty after clicking a user in the chat list.

10.3 Chat Input & Controls

- **Issue 5 — Emoji and attachment buttons:** these buttons in the message input bar are not functional.
- **Issue 6 — Chat options menu unresponsive:** the three-dot menu options (View Profile, Mute Notifications, Delete Chat) do not respond to clicks.
- **Issue 7 — Group creation missing:** there is currently no way to create group chats.
- **Issue 8 — Send/arrow button misplaced:** the send (arrow) button is positioned in the middle of the input row instead of at the end.

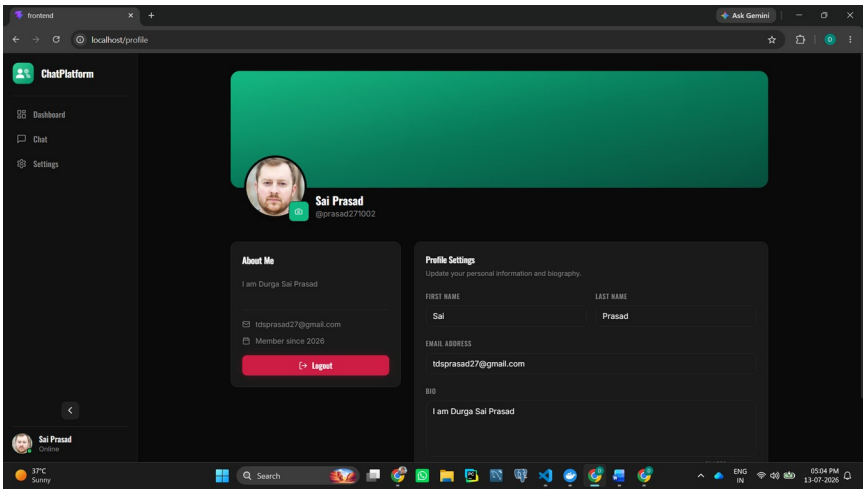

Figure 4: Send/arrow button positioned in the middle of the input row.

10.4 Notifications

- **Issue 9 — Missing message notifications:** no notification is shown to the user when a new message is received.

10.5 Summary Table

#	Area	Issue
1	Message Display	Latest message shown at top instead of bottom (WhatsApp-style ordering expected)
2	Message Display	Latest message intermittently fails to display correctly
3	Dashboard	Recent chats list does not show the latest message
4	Navigation	Selecting a user clears the chat view; refresh restores it
5	Chat Input	Emoji and attachment buttons non-functional
6	Chat Input	Three-dot menu (Profile, Mute, Delete Chat) unresponsive
7	Feature Gap	No option to create group chats
8	Chat Input	Send/arrow button placed in the middle instead of at the end
9	Notifications	No notification shown on receiving new messages

11. Conclusion

The Real-Time Chat Platform project is structured to give every team member clear ownership of a module while ensuring the pieces integrate cleanly through a shared API contract, design system, and Git branching strategy. By following the milestone plan and prioritizing MVP features first, the team can deliver a functional, secure, and scalable one-to-one messaging application within the internship timeline, with a clear, documented path for future enhancements such as group chat and end-to-end encryption.